

Two Pointers

Sean

May 6, 2026

Outline

- When to think two pointers
- Opposite-direction pointers
- Sliding window
- Invariants and common mistakes



Introduction

- Two pointers is a way to scan a range while keeping only a small amount of state



Introduction

- Two pointers is a way to scan a range while keeping only a small amount of state
- In CP, it often turns brute force from $\mathcal{O}(n^2)$ into $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$



Introduction

- Two pointers is a way to scan a range while keeping only a small amount of state
- In CP, it often turns brute force from $\mathcal{O}(n^2)$ into $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$
- The main idea is to move pointers only when doing so preserves a useful invariant



- 1 When to Think Two Pointers
- 2 Opposite-Direction Pointers
- 3 Sliding Window
- 4 Invariants and Common Mistakes



When to Think Two Pointers

When to Think Two Pointers

- We are scanning a contiguous range or a sorted array



When to Think Two Pointers

- We are scanning a contiguous range or a sorted array
- After moving one pointer, we usually do not need to move it backward



When to Think Two Pointers

- We are scanning a contiguous range or a sorted array
- After moving one pointer, we usually do not need to move it backward
- A condition becomes easier or harder in one direction



When to Think Two Pointers

- We are scanning a contiguous range or a sorted array
- After moving one pointer, we usually do not need to move it backward
- A condition becomes easier or harder in one direction
- We want to avoid checking every pair or every subarray independently



Why It Helps

- Brute force often restarts work from scratch for each left endpoint

Why It Helps

- Brute force often restarts work from scratch for each left endpoint
- Two pointers reuses information from the previous step



Why It Helps

- Brute force often restarts work from scratch for each left endpoint
- Two pointers reuses information from the previous step
- Each pointer usually moves at most n times

Why It Helps

- Brute force often restarts work from scratch for each left endpoint
- Two pointers reuses information from the previous step
- Each pointer usually moves at most n times
- That is why many two-pointer solutions run in linear time after sorting



- 1 When to Think Two Pointers
- 2 Opposite-Direction Pointers
- 3 Sliding Window
- 4 Invariants and Common Mistakes



Opposite-Direction Pointers

Sorted Array Pair Search

- A classic setup is a sorted array with one pointer at each end



Sorted Array Pair Search

- A classic setup is a sorted array with one pointer at each end
- Compare the current pair to the target



Sorted Array Pair Search

- A classic setup is a sorted array with one pointer at each end
- Compare the current pair to the target
- If the sum is too small, move the left pointer right



Sorted Array Pair Search

- A classic setup is a sorted array with one pointer at each end
- Compare the current pair to the target
- If the sum is too small, move the left pointer right
- If the sum is too large, move the right pointer left



Two Sum on a Sorted Array

```
1  int l = 0, r = n - 1;
2  while (l < r) {
3      int s = a[l] + a[r];
4      if (s == target) return true;
5      if (s < target) l++;
6      else r--;
7  }
8  return false;
```



Why the Moves Are Correct

- The array is sorted, so moving ℓ right can only increase the sum



Why the Moves Are Correct

- The array is sorted, so moving l right can only increase the sum
- Moving r left can only decrease the sum



Why the Moves Are Correct

- The array is sorted, so moving l right can only increase the sum
- Moving r left can only decrease the sum
- If $a[l] + a[r]$ is too small, no pair using that l can work



Why the Moves Are Correct

- The array is sorted, so moving l right can only increase the sum
- Moving r left can only decrease the sum
- If $a[l] + a[r]$ is too small, no pair using that l can work
- If it is too large, no pair using that r can work



Typical Opposite-Pointer Problems

- Two sum in a sorted array



Typical Opposite-Pointer Problems

- Two sum in a sorted array
- Count pairs with sum less than or equal to k



Typical Opposite-Pointer Problems

- Two sum in a sorted array
- Count pairs with sum less than or equal to k
- Find whether two sorted arrays contain a pair meeting a target condition



Typical Opposite-Pointer Problems

- Two sum in a sorted array
- Count pairs with sum less than or equal to k
- Find whether two sorted arrays contain a pair meeting a target condition
- Partition-style scans where one pointer moves inward from each side



Example: Sum of Two Values

CSES: Sum of Two Values

Given an array and a target x , find two different positions whose values sum to x .

Example: Sum of Two Values

CSES: Sum of Two Values

Given an array and a target x , find two different positions whose values sum to x .

- Store each value together with its original index

Example: Sum of Two Values

CSES: Sum of Two Values

Given an array and a target x , find two different positions whose values sum to x .

- Store each value together with its original index
- Sort by value, then run opposite-direction pointers



Example: Sum of Two Values

CSES: Sum of Two Values

Given an array and a target x , find two different positions whose values sum to x .

- Store each value together with its original index
- Sort by value, then run opposite-direction pointers
- If the current sum is too small, move left; if too large, move right



Example: Sum of Two Values

CSES: Sum of Two Values

Given an array and a target x , find two different positions whose values sum to x .

- Store each value together with its original index
- Sort by value, then run opposite-direction pointers
- If the current sum is too small, move left; if too large, move right
- If the sum matches, output the original indices



1 When to Think Two Pointers

2 Opposite-Direction Pointers

3 Sliding Window

4 Invariants and Common Mistakes



Sliding Window

Sliding Window

- Sometimes both pointers move from left to right

Sliding Window

- Sometimes both pointers move from left to right
- The interval $[l, r]$ is the current window



Sliding Window

- Sometimes both pointers move from left to right
- The interval $[l, r]$ is the current window
- Expand the window by moving r



Sliding Window

- Sometimes both pointers move from left to right
- The interval $[l, r]$ is the current window
- Expand the window by moving r
- Shrink the window by moving l until the invariant is restored



Window Invariant

- The invariant is the rule that must stay true for the current window

Window Invariant

- The invariant is the rule that must stay true for the current window
- Example: “the sum of the window is at most k ”



Window Invariant

- The invariant is the rule that must stay true for the current window
- Example: “the sum of the window is at most k ”
- Example: “all values in the window are distinct”



Window Invariant

- The invariant is the rule that must stay true for the current window
- Example: “the sum of the window is at most k ”
- Example: “all values in the window are distinct”
- Every pointer move should either restore or preserve that rule



Longest Window with Sum $\leq k$

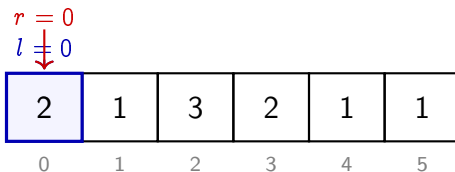
For a nonnegative array:

```
1  int l = 0;
2  long long sum = 0;
3  int best = 0;
4  for (int r = 0; r < n; r++) {
5      sum += a[r];
6      while (sum > k) {
7          sum -= a[l];
8          l++;
9      }
10     best = max(best, r - l + 1);
11 }
```



Sliding-Window Trace

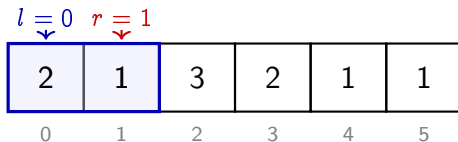
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Window $[0, 0]$, sum = 2, valid

Sliding-Window Trace

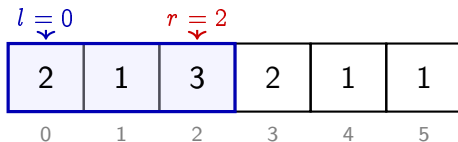
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[0, 1]$, sum = 3, still valid

Sliding-Window Trace

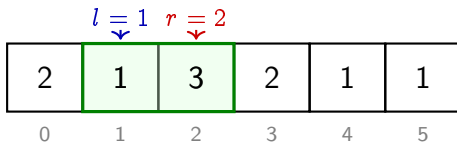
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[0, 2]$, sum = 6, too large

Sliding-Window Trace

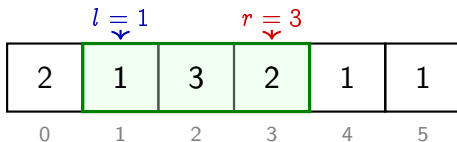
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Shrink left: window $[1, 2]$, sum = 4, valid again

Sliding-Window Trace

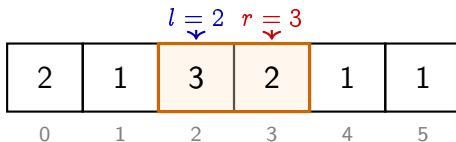
Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Expand right: window $[1, 3]$, sum = 6, too large again

Sliding-Window Trace

Let $a = [2, 1, 3, 2, 1, 1]$ and $k = 5$.



Shrink left: window $[2, 3]$, sum = 5, valid

How the Window Moves

- Add the new right endpoint into the current state

How the Window Moves

- Add the new right endpoint into the current state
- If the window becomes invalid, move $\perp$ until it is valid again

How the Window Moves

- Add the new right endpoint into the current state
- If the window becomes invalid, move ℓ until it is valid again
- Once the window is valid, update the answer



How the Window Moves

- Add the new right endpoint into the current state
- If the window becomes invalid, move l until it is valid again
- Once the window is valid, update the answer
- Because l and r only move forward, the total work is linear



When Sliding Window Works

- The window condition should behave monotonically as the window grows or shrinks



When Sliding Window Works

- The window condition should behave monotonically as the window grows or shrinks
- If expanding makes the condition harder to satisfy, shrinking can restore it



When Sliding Window Works

- The window condition should behave monotonically as the window grows or shrinks
- If expanding makes the condition harder to satisfy, shrinking can restore it
- This is why nonnegative sums work well with sliding windows



When Sliding Window Works

- The window condition should behave monotonically as the window grows or shrinks
- If expanding makes the condition harder to satisfy, shrinking can restore it
- This is why nonnegative sums work well with sliding windows
- With arbitrary negative values, the same strategy often breaks



When Sliding Window Fails

- Suppose $a = [5, -4, 3]$ and we want the longest subarray with sum ≤ 4



When Sliding Window Fails

- Suppose $a = [5, -4, 3]$ and we want the longest subarray with sum ≤ 4
- The whole array has sum 4, so the correct answer has length 3



When Sliding Window Fails

- Suppose $a = [5, -4, 3]$ and we want the longest subarray with sum ≤ 4
- The whole array has sum 4, so the correct answer has length 3
- But a simple sliding window sees $5 > 4$ first, so it removes the left endpoint too early



When Sliding Window Fails

- Suppose $a = [5, -4, 3]$ and we want the longest subarray with sum ≤ 4
- The whole array has sum 4, so the correct answer has length 3
- But a simple sliding window sees $5 > 4$ first, so it removes the left endpoint too early
- Negative values can make the sum go down again later, so the window condition is no longer monotone



Example: Playlist

CSES: Playlist

Find the longest contiguous subarray in which all values are distinct.

Example: Playlist

CSES: Playlist

Find the longest contiguous subarray in which all values are distinct.

- Expand the right endpoint while tracking which values are in the window

Example: Playlist

CSES: Playlist

Find the longest contiguous subarray in which all values are distinct.

- Expand the right endpoint while tracking which values are in the window
- If a duplicate appears, move the left endpoint until the window is distinct again

Example: Playlist

CSES: Playlist

Find the longest contiguous subarray in which all values are distinct.

- Expand the right endpoint while tracking which values are in the window
- If a duplicate appears, move the left endpoint until the window is distinct again
- Update the best window length after restoring the invariant



Example: Playlist

CSES: Playlist

Find the longest contiguous subarray in which all values are distinct.

- Expand the right endpoint while tracking which values are in the window
- If a duplicate appears, move the left endpoint until the window is distinct again
- Update the best window length after restoring the invariant
- A set or frequency map can maintain the window state



- 1 When to Think Two Pointers
- 2 Opposite-Direction Pointers
- 3 Sliding Window
- 4 Invariants and Common Mistakes



Invariants and Common Mistakes

Common Mistakes

- Forgetting to move a pointer, which can create an infinite loop

Common Mistakes

- Forgetting to move a pointer, which can create an infinite loop
- Updating the answer before the window becomes valid



Common Mistakes

- Forgetting to move a pointer, which can create an infinite loop
- Updating the answer before the window becomes valid
- Using sliding window when the condition is not monotonic



Common Mistakes

- Forgetting to move a pointer, which can create an infinite loop
- Updating the answer before the window becomes valid
- Using sliding window when the condition is not monotonic
- Off-by-one errors in the window length $r - l + 1$



Choosing the Right Pattern

- Use opposite-direction pointers when the array is sorted and the comparison is about both ends



Choosing the Right Pattern

- Use opposite-direction pointers when the array is sorted and the comparison is about both ends
- Use a sliding window when both pointers naturally move forward



Choosing the Right Pattern

- Use opposite-direction pointers when the array is sorted and the comparison is about both ends
- Use a sliding window when both pointers naturally move forward
- Ask what invariant should stay true while the pointers move



Choosing the Right Pattern

- Use opposite-direction pointers when the array is sorted and the comparison is about both ends
- Use a sliding window when both pointers naturally move forward
- Ask what invariant should stay true while the pointers move
- If you cannot justify pointer movement, the method is probably wrong



Takeaways

- Two pointers is about reusing work instead of restarting from scratch
- Sorted arrays often lead to opposite-direction pointers
- Range problems often lead to sliding windows
- The invariant is the real reason the pointer moves are correct



The End

Thanks for listening.

© 2026 Sean. Please do not repost or redistribute without permission.

